

En esta etapa del proyecto se realizó el desarrollo de los distintos componentes que integran el sistema de mi equipo AMP para una SOFOM (socio formador). El propósito fue construir las funcionalidades necesarias para que el sistema pudiera operar de forma organizada y correcta. Permitiendo la interacción entre usuarios, vistas, lógica del sistema y almacenamiento de datos.

El desarrollo se realizó utilizando una arquitectura que separa responsabilidades entre componentes, facilitando el mantenimiento del código y permitiendo futuras mejoras sin afectar el funcionamiento general del sistema.

Objetivo:

Desarrollar e integrar los módulos del sistema necesarios para cubrir los requerimientos funcionales del proyecto, garantizando comunicación entre interfaz, lógica de negocio y base de datos.

Componentes desarrollados

Interfaz de usuario:

Se crearon las vistas del sistema usando EJS, HTML, CSS y JS. Estas permiten al usuario iniciar sesión, navegar entre páginas y utilizar las funciones principales del sistema.

Lógica del sistema

Se desarrollaron rutas y controladores con [Node.js](#) y Express. Estos componentes se encargan de recibir las solicitudes de usuarios, procesarlas y enviar una respuesta adecuada.

Base de datos

Se integró PostgreSQL para guardar y consultar información del sistema. La conexión se realizó mediante configuración segura y uso de variables de entorno.

Integración

Los componentes fueron conectados para que el sistema funcionara de forma completa. La interfaz envía datos al servidor, el servidor procesa la información y la base de datos responde con los datos.

Pruebas realizadas

Se probaron funciones como inicio de sesión, navegación entre vistas, control de roles, conexión con la base de datos y el cierre de sesión. En general, los componentes funcionan correctamente después de realizar algunos ajustes.

Problemas encontrados

Como lo comente anteriormente, tuvimos ese problemita con la dashboard pero otro problema que tuvimos fue con las cookies porque al momento de hacer login nos marcaba error y además los botones de "Aceptar me gustan las galletitas" y "No me gustan las galletitas" no tenían funcionalidad, si aparecían pero al momento de dar click no se reportaba la acción.

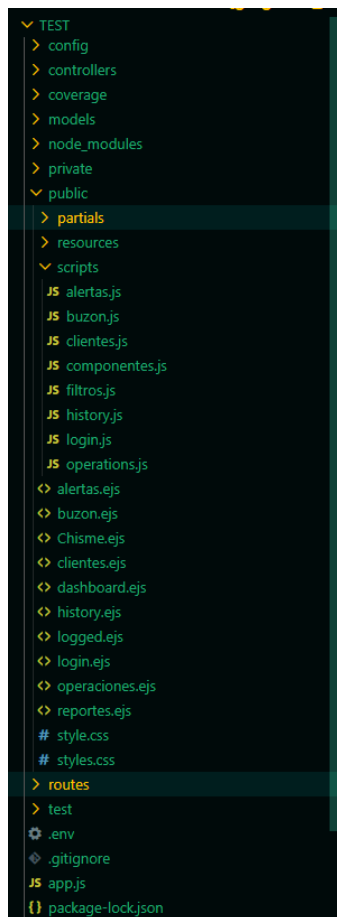
Mucho antes de esos problemas tuvimos un problemita con login y sign in ya que cada apartado chocaba entre sí y además no se registraba en la base de datos y mandaba error. Actualmente ya están corregidos estos errores.

Convención de código del equipo

El equipo definió una convención de código para el proyecto:

Las principales reglas utilizadas fueron:

- Organización del proyecto utilizando arquitectura MVC
- Uso de nombres descriptivos para variables, funciones y archivos
- Uso de camelCase para variables y funciones
- Separación de responsabilidades entre rutas, controladores, modelos y vistas
- Uso de archivos .env para información sensible
- Mantener formato y estructura uniforme en todos los módulos



La aplicación de estas convenciones permitió mejorar la legibilidad del código y facilitar el mantenimiento del sistema.

Contribuciones individuales de front-end en el repositorio del proyecto

Las contribuciones realizadas en la parte de front-end estuvieron enfocadas en la construcción de la interfaz y en la experiencia del usuario.

- Desarrollo de pantallas de inicio de sesión y registro

- Diseño e integración del dashboard principal.

- Aplicación de estilos mediante CSS

- Configuración de navegación entre vistas

- Integración de vistas dinámicas usando EJS

Estas contribuciones permitieron mejorar la interacción del usuario con el sistema y mantener una interfaz funcional y organizada

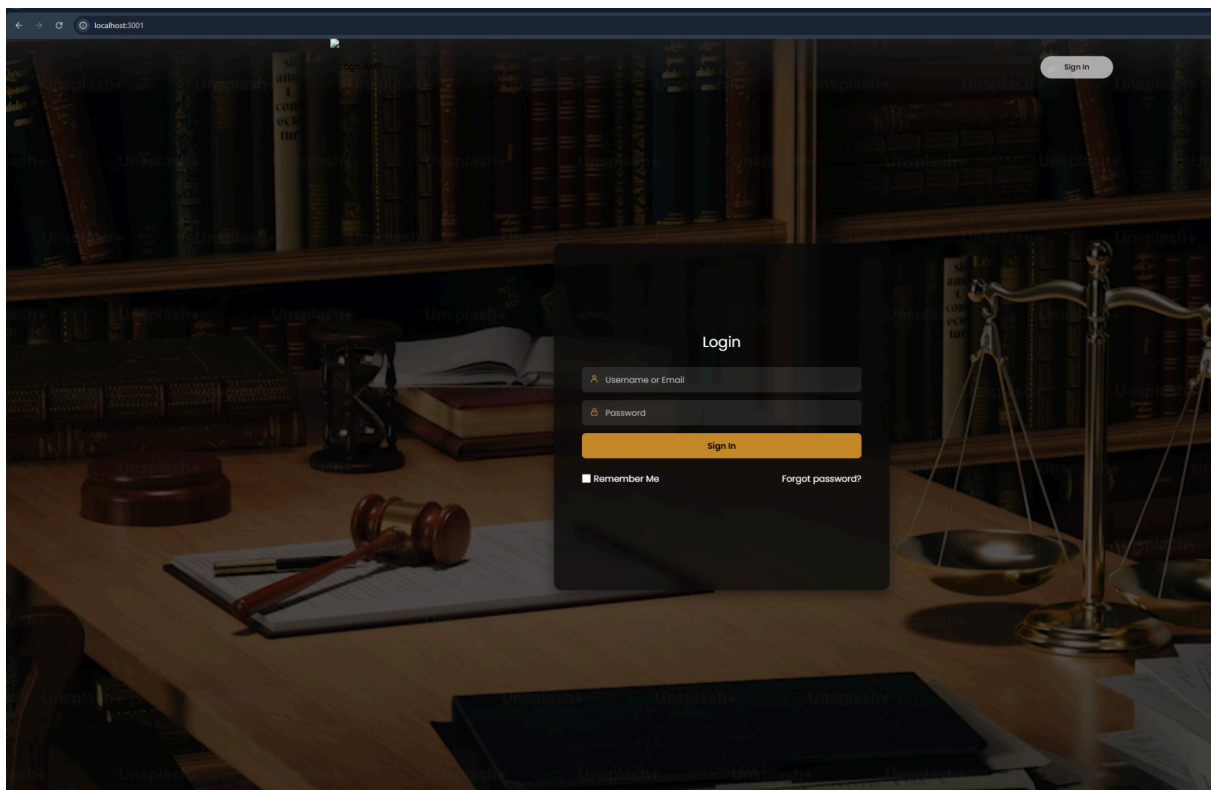


Figura 1. Visualización del login

```
login.ejs TEST\public\login.ejs |html|body.login-page|div.wrapper|div.nav|div.nav-button|button.loginBtn.btn.white-btn
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link href="https://unpkg.com/boxicons@2.1.4/css/boxicons.min.css" rel="stylesheet">
7   <link rel="stylesheet" href="/style.css">
8
9   <title>AMP | Login & Registration</title>
10 </head>
11
12 <body class="login-page">
13
14   <div class="wrapper">
15     <nav class="nav">
16       <div class="nav-logo">
17         
18       </div>
19
20       <div class="nav-button">
21         <button type="button" class="btn white-btn" id="loginBtn">Sign In</button>
22         <button type="button" class="btn" id="registerBtn">Sign Up</button>
23       </div>
24     </nav>
25
26     <div class="form-box">
27
28       <form class="login-container" id="login" action="/usuarios/login" method="POST">
29         <input type="hidden" name="_csrf" value="{% csrf_token %}">
30         <div class="top">
31           <header>Login</header>
32         </div>
33
34         <div class="input-box">
35           <input type="text" name="username" class="input-field" placeholder="Username or Email" required>
36           <i class="bx bx-user"></i>
37         </div>
38
39       </form>
40     </div>
41   </div>
42 </body>
43 </html>
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

PS C:\Users\David\Downloads\TEST935PM29MAY>

Figura 2. Visualización de login.ejs

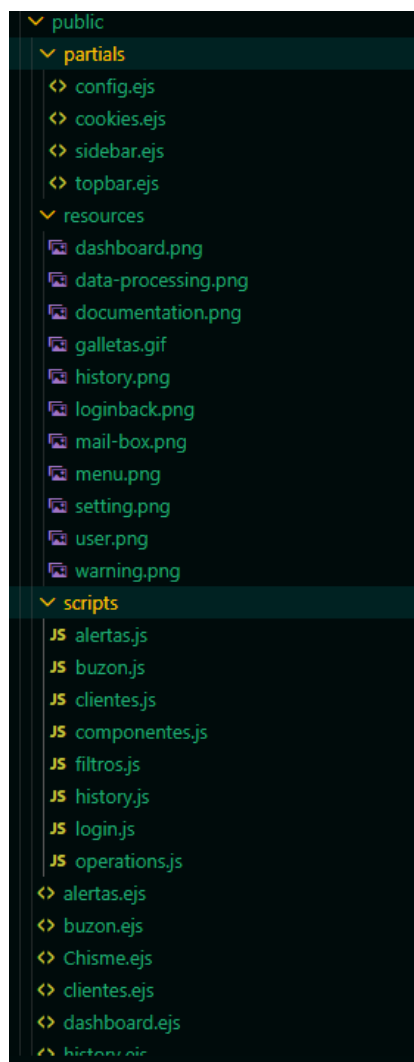


Figura 3. Visualización de la estructura del proyecto

```
# style.css TEST\publicstyle.css...
1 @import url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&display=swap');
2
3 * {
4   margin: 0;
5   padding: 0;
6   box-sizing: border-box;
7   font-family: 'Poppins', sans-serif;
8 }
9
10 /* fondo con overlay oscuro para que el texto respire */
11 body {
12   min-height: 100vh;
13   background:
14     linear-gradient(rgba(0, 0, 0, 0.65), rgba(0, 0, 0, 0.65)),
15     url("../resources/loginback.png");
16   background-size: cover;
17   background-position: center;
18   background-repeat: no-repeat;
19   background-attachment: fixed;
20   overflow: hidden;
21 }
22
23 .wrapper {
24   display: flex;
25   justify-content: center;
26   align-items: center;
27   min-height: 100vh;
28   background: transparent;
29 }
30
31 /* nav fija con degradado que se funde hacia abajo */
32 .nav {
33   position: fixed;
34   top: 0;
35   display: flex;
36   justify-content: space-around;
37   width: 100%;
38   height: 100px;
```

Figura 4. Visualización del CSS

Contribuciones individuales de back-end en el repositorio del proyecto

Las contribuciones realizadas en el back-end estuvieron enfocadas en el desarrollo de la lógica del sistema y comunicación con la base de datos

Actividades realizadas:

- Configuración del servidor usando [Node.js](#) y Express
- Desarrollo de rutas y controladores.
- Implementación del sistema de autenticación
- Configuración y manejo de sesiones
- Integración con PostgreSQL
- Implementación de validaciones y control de acceso por roles

Estas actividades permitieron garantizar el funcionamiento interno del sistema y la correcta comunicación entre la interfaz y la base de datos.

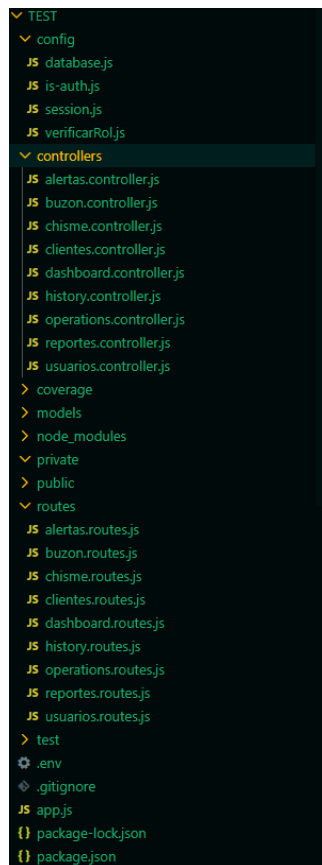


Figura 5. Estructura del repo

```
JS app.js TEST\app.js\...
1  require("dotenv").config();
2
3  const express = require("express");
4  const path = require("path");
5  //const session = require("express-session");
6  const sessionConfig = require("../config/session");
7  const cookieParser = require("cookie-parser");
8  const cors = require("cors");
9  const helmet = require("helmet");
10 const fs = require("fs");
11
12 const { doubleCsrf } = require("csrf-csrf");
13
14 const app = express();
15
16 /* =====
17 | | CREAM CARPETA PRIVATE
18 ===== */
19
20 fs.mkdirSync(
21   path.join(__dirname, "private"),
22   { recursive: true }
23 );
24
25 /* =====
26 | | CONFIGURACIÓN EJS
27 ===== */
28
29 app.set("view engine", "ejs");
30
31 app.set(
32   "views",
33   path.join(__dirname, "public")
34 );
35
36 /* =====
37 | | MIDDLEWARES
38 ===== */
```

Figura 6. Visualización de app.js

```
1 const express = require("express");
2 const router = express.Router();
3 const dashboardController = require("../controllers/dashboard.controller");
4 const isAuth = require("../config/is-auth");
5
6 router.get("/", (req, res) => res.redirect("/dashboard"));
7 router.get("/dashboard", isAuth, dashboardController.renderDashboard);
8
9 module.exports = router;
```

Figura 7. Visualización de routes

```
1 require("dotenv").config();
2 router = require("express")
3 const { Pool } = require("pg");
4
5 const pool = new Pool({
6   connectionString: process.env.DATABASE_URL,
7   ssl: {
8     rejectUnauthorized: false
9   },
10   max: 5,
11 });
12
13 module.exports = pool;
```

Figura 9. Visualización de database.js

Conclusión

El desarrollo de componentes permitió construir una base funcional para nuestro sistema. Gracias a la separación entre interfaz, lógica y datos, el proyecto quedó mejor organizado y listo para seguir mejorándolo.